

PMTAC

APG Rate Analyzer

Technical Documentation

Architecture · Build · Azure Trial Deployment

Version: 1.0 · Beta deployment

Date: May 2026

Prepared for: PMTAC IT and Engineering

Stack: .NET 10 · ASP.NET Core MVC · EF Core 10 · Azure SQL · Azure App Service

ISO 27001:2022 Certified · © 2026 PMTAC

Table of Contents

1. Executive Summary.....	5
Project goals.....	5
What this document covers.....	5
2. Architecture Overview	6
2.1 Technology stack.....	6
2.2 High-level architecture.....	6
2.3 Cross-cutting concerns.....	7
3. Domain Model	8
3.1 Core entities.....	8
3.2 Reference data (loaded from NYS DOH / eMedNY workbooks).....	8
3.3 Operational entities	8
3.4 NYS APG methodology — the math.....	9
3.5 The 837/835 link	9
4. Database Schema.....	10
4.1 Migration history	10
4.2 Indexes	10
4.3 Cascade behavior	11
5. Build Phases	12
5.1 Phase 1 — Skeleton	12
5.2 Phase 2 — Reference data loaders	12
5.3 Phase 3 — APG calculation engine	12
5.4 Phase 4 — EDI parsers and upload pipeline	13
Parsers.....	13
Upload simplification (later session).....	13
Upload service.....	13
5.5 Phase 5 — Analytics and exports	13
Analytics service.....	13
Exports	13
5.6 Phase 6 — Polish, theming, deployment readiness.....	14
5.7 Post-go-live extensions	14
5.7.1 User management + role-based authorization.....	14
5.7.2 Per-user data isolation	14
5.7.3 CMS Medicare PFS integration	14

5.7.4 Multi-line Rate Calculator	15
5.7.5 Analytics dashboard	15
6. Security and Access Control	16
6.1 Authentication	16
6.2 Authorization — role matrix	16
6.3 Role enforcement	16
6.4 Per-user data isolation	17
6.5 The bootstrap-admin problem	17
7. CMS Medicare PFS Integration	19
7.1 Data source	19
7.2 Formula	19
7.3 Caching strategy	19
7.4 Cache refresh	20
7.5 Outbound network requirement	20
7.6 Threading caveat	20
8. Azure Trial Deployment	21
8.1 Prerequisites	21
8.2 Code-level preparation	21
8.3 Step C1 — Create the free Azure SQL Database	21
8.4 Step C2 — Create the App Service	22
8.5 Step C2b — Wire up the connection string	23
8.6 Step C2c — SQL firewall rules	23
8.7 Step C3 — Apply database migrations	24
8.8 Step D — Bootstrap the first admin	24
8.9 Step D2 — Load reference data and create beta accounts	25
8.10 Step D3 — Smoke tests	25
9. Operational Notes	26
9.1 Free-tier characteristics and trade-offs	26
9.2 Cold-start playbook	26
9.3 Quarterly CMS refresh	27
9.4 Reference data refresh	27
9.5 Backups	27
10. Roadmap and Deferred Items	28
10.1 Cosmetic / quick wins	28
10.2 Feature work deferred from V1	28

10.3 V3 — separate analytics application.....	28
10.4 Production deployment (handed to IT)	28
11. Appendices.....	30
11.1 Glossary.....	30
11.2 Key file locations	31
11.3 Standard error references.....	32
11.4 Useful Azure CLI commands	32
11.5 Document revision history.....	33

1. Executive Summary

The APG Rate Analyzer is a web application built for PMTAC (a Medicaid Article 28 compliance and analytics consultancy) to inspect, validate, and audit medical claim reimbursements against the New York State Department of Health (NYS DOH) ambulatory patient group (APG) methodology. The application ingests X12 EDI 835 (electronic remittance advice) and 837 (claim submission) files, parses each claim, calculates the expected APG payment using NYS DOH reference data, and reports the variance between what should have been paid and what was actually paid.

Beyond APG analytics, the application also computes the equivalent CMS Medicare Physician Fee Schedule (PFS) rate for each procedure, allowing PMTAC analysts to compare what a professional service would pay under Medicare against what was paid under NYS Medicaid.

Project goals

- **Replace** an internal Python prototype with a production-ready .NET application IT can deploy and maintain.
- **Calculate** correct APG payments using the NYS DOH-published reference workbooks (HCPCS crosswalks, weights, base rates, fee schedule).
- **Audit** payer behavior — flag underpaid claims, identify CARC denial patterns, and surface compression by procedure / EAPG / peer group.
- **Compare** payments against CMS Medicare PFS for cross-program benchmarking.
- **Isolate** data per analyst — each user sees only their own uploaded claims; admins have organization-wide visibility.
- **Deploy** to Microsoft Azure for both production (managed by IT) and a free-tier beta environment (managed by the project lead).

What this document covers

- The complete technical architecture of the application — runtime stack, domain model, database schema, security model, and external integrations.
- The build phases and the rationale behind each decision — useful for any developer inheriting the codebase.
- The full Azure trial-deployment playbook — every step from creating the subscription to handing the URL to beta testers.
- Operational notes — common failure modes, recovery procedures, and the trade-offs between Azure free, Basic, and Standard tiers.
- A roadmap of future work and deferred items.

Audience

This document is technical but should be readable by domain experts (medical billing analysts) who want to understand *what* the system does. Deep .NET internals are confined to the "Architecture" and "Build phases" sections, which can be skipped on a first pass.

2. Architecture Overview

The APG Rate Analyzer is a server-rendered web application following the canonical ASP.NET Core MVC pattern. There is no separate API layer or single-page front-end; each user interaction is a standard HTTP form post or GET request, and the server returns fully-rendered Razor HTML. This deliberately keeps the architecture simple and aligns with PMTAC IT's existing skill set.

2.1 Technology stack

Layer	Technology	Version	Why
Runtime	.NET	10.0 (LTS)	Long-term support, mature, native to the Azure stack PMTAC IT manages
Web framework	ASP.NET Core MVC	10.0	Server-rendered Razor; least JavaScript surface area
ORM	Entity Framework Core	10.0	Code-first migrations, LINQ-to-SQL, no hand-written ADO.NET
Database (dev)	SQL Server LocalDB	Latest	Zero-install local dev; ships with Visual Studio
Database (prod/beta)	Azure SQL Database	Serverless / Basic	Managed, free-tier eligible, identical T-SQL dialect
Identity	ASP.NET Core Identity	10.0	Cookie auth, role-based authorization, password hashing
CSS	Bootstrap	5.x	Grid, components, dark/light theming hooks
Charts	Chart.js	4.4.6 (CDN)	Pure browser; no Node tooling on server
Excel	ClosedXML	0.x	.xlsx read/write without Office interop
Legacy .xls	NPOI	2.x	BIFF reader for older NYS DOH workbooks
PDF	QuestPDF (Community)	2024.x	Layout-engine PDF generation; UB-04 / CMS-1500 forms
Hosting	Azure App Service (Windows)	F1 free / B1 basic	Managed IIS-equivalent, integrated with VS Publish wizard
Build tooling	Visual Studio 2022 / dotnet CLI	Latest	Both work; Publish wizard for one-click deploys

2.2 High-level architecture

The system has three logical tiers that all run inside the same App Service worker process:

- **Web / UI tier** — Razor views rendered server-side. Bootstrap 5 + custom PMTAC theme. Chart.js loaded from CDN for analytics.
- **Service / engine tier** — APG calculation engine, CMS Medicare engine, EDI parsers, claim linker, exporters. Pure C# with dependency-injected database access.
- **Persistence tier** — SQL Server (LocalDB in dev, Azure SQL in prod) accessed exclusively through EF Core. No stored procedures; all logic lives in C#.

External dependencies are limited to a single outbound integration: the CMS PFS DKAN datastore at `pfs.data.cms.gov`, used live by the CMS rate engine.

2.3 Cross-cutting concerns

- **Authentication** — cookie-based Identity. No JWT, no OAuth, no SSO yet (deferred until IT decides between corporate Entra ID and standalone).
- **Authorization** — role-based (`[Authorize(Roles="admin")]` etc.). Three roles: admin, analyst, viewer.
- **Data isolation** — every business-relevant entity carries an `OwnerUserId`. Read queries are filtered through an `OwnedBy()` extension method based on the current user's effective scope.
- **Logging** — `Microsoft.Extensions.Logging` at Information / Warning / Error. Console output is captured by Azure App Service Log Stream.
- **Configuration** — `appsettings.json` for non-secret defaults; Azure App Service Configuration for the connection string and any production-only overrides.
- **Caching** — none in V1 except the in-process CMS catalog cache and a 24-hour DB cache for CMS rates. Reference data is loaded into SQL once and queried per-request.

3. Domain Model

The domain centers on a single concept: a **claim** — one CLP segment from an X12 EDI file, representing one patient encounter being submitted to or remitted by a payer. Every other entity hangs off the claim or is reference data used to price it.

3.1 Core entities

Entity	Table	Cardinality	Purpose
ParsedClaim	parsed_claim	1	Claim header — CLP01, dates, parties, totals, FK to APG result
ParsedServiceLine	parsed_service_line	N per claim	SVC / SV1 / SV2 lines — one row per CPT/HCPCS line
ClaimAdjustment	claim_adjustment	N per claim	CAS rows — both claim-level and line-level adjustments (CARC codes)
ApgResultRecord	apg_result	1 per claim	Cached engine output: correct payment, variance, line-by-line math

3.2 Reference data (loaded from NYS DOH / eMedNY workbooks)

Entity	Table	Approx. rows	Source
HcpcsToEapg	hcpcs_to_eapg	~21,000	eMedNY APG Crosswalk
Icd10ToEapg	icd10_to_eapg	~75,000	eMedNY APG Crosswalk
ApgWeight	apg_weights	~21,000	NYS DOH weights history
ApgBaseRate	apg_base_rates	~180	PMTAC Fee Calculator (or NYS DOH DTC inventory)
PxBasedWeight	px_based_weights	~5,300	NYS DOH weights history
FeeScheduleItem	fee_schedule	~2,100	NYS DOH weights history
ProviderCounty	provider_county	62	PMTAC Fee Calculator (NY counties → region)

3.3 Operational entities

Entity	Table	Purpose
ProviderConfig	provider_config	Active provider (per user) — peer group, county, region, CMS locality

Entity	Table	Purpose
CmsRateCache	cms_rate_cache	24h cache of CMS PFS rate lookups (HCPCS × modifier × locality × year)
AspNetUsers / AspNetRoles	AspNet*	Standard ASP.NET Core Identity tables for auth

3.4 NYS APG methodology — the math

The APG calculation determines the "correct" payment for a Medicaid Article 28 outpatient claim using a priority ladder. For each service line:

1. **Fee Schedule** check — if the procedure code appears in the NYS DOH fee schedule for the date of service, use the flat fee (typically applies to packaged drugs and supplies).
2. **Procedure-based weight** check — if the HCPCS has an explicit Px weight on file, compute weight × base_rate.
3. **APG weight** check — fall through to APG-level weight (the procedure's assigned EAPG's weight) × base_rate.
4. **Visit-purpose ICD override** — for incidental E/M codes (99201–99499), if the principal diagnosis maps to a different EAPG that pays more, use that instead. This is the rule that produces the canonical "\$132.09" result for a 99213 + E11.9 encounter.
5. **Modifiers** — the U6 modifier multiplies by 1.5×; multi-procedure discounting reduces secondary lines by 50%; capital add-on fee is added to the claim total when the provider is eligible.

The base rate is selected by joining the active provider's peer group (e.g., "Clinic*", "Amb Surg", "Renal") with their region (Upstate / Downstate, derived from the configured county) and the date of service. The same procedure can have different base rates over time as NYS DOH publishes updates.

3.5 The 837/835 link

Claims arrive in two complementary formats: the ****837**** (the provider submitting the claim) and the ****835**** (the payer's remittance with paid amounts and adjustments). When both land in the database for the same CLP01 / CLM01 ID, the application links them via ****ParsedClaim.LinkedClaimIdFk**** — a bidirectional self-FK. Linking enriches the 835 with diagnosis codes from the 837 (835s rarely carry dx codes), which lets the visit-purpose ICD override fire correctly when re-pricing.

4. Database Schema

The database is generated by EF Core code-first migrations. Every schema change is a numbered migration in ``APGAnalyzer/Data/Migrations/``. The current set is listed below in chronological order — applying them all (``dotnet ef database update``) brings a fresh database to the current schema.

4.1 Migration history

Migration	Purpose
0000000000000000_CreatelIdentitySchema	ASP.NET Core Identity tables (AspNetUsers, AspNetRoles, AspNetUserRoles, etc.)
20260429150458_AddDomainTables	Reference tables: hcpcs_to_eapg, icd10_to_eapg, apg_weights, etc.
20260429194457_AddProviderConfig	Active provider configuration table
20260429200242_ProviderCountyNoIdentity	Drop IDENTITY column on provider_county (county codes are externally assigned)
20260429204928_AddClaimTables	Operational tables: parsed_claim, parsed_service_line, claim_adjustment, apg_result
20260501140008_AddOwnerUserId	Per-user data isolation: OwnerUserId column on parsed_claim and provider_config + backfill SQL
20260501142326_AddCmsRateCache	CMS Medicare PFS rate cache table

4.2 Indexes

Indexes are defined in the DbContext OnModelCreating method. Notable ones:

- **ix_hcpcs_eapg_code_date** on (Hcpcs, QuarterEffectiveDate) — primary lookup for procedure → EAPG mapping
- **ix_icd10_eapg_code_date** on (DxCd, EffectiveDate) — diagnosis → EAPG (visit-purpose override)
- **uq_apg_weight_date** on (Apg, EffectiveDate), unique — at most one weight per APG per effective date
- **ix_base_rate_lookup** on (Source, PeerGroup, Region, EffectiveDate) — join target during pricing
- **ix_parsed_claim_owner_created** on (OwnerUserId, CreatedAt) — drives the Claims list page (sorted desc by CreatedAt, scoped per user)
- **ix_provider_config_owner_active** on (OwnerUserId, IsActive) — single row hot-path read at every claim calculation
- **uq_cms_rate_cache_key** on (Hcpcs, Modifier, Locality, Year), unique — ensures one cache row per parameter set

4.3 Cascade behavior

Foreign keys behave as follows when a parent is deleted:

Parent	Child	On parent delete
parsed_claim	parsed_service_line	CASCADE — lines are useless without their claim
parsed_claim	claim_adjustment	CASCADE — same reason
parsed_claim	apg_result	CASCADE — cached output goes with the claim
parsed_claim (linked sibling)	parsed_claim	NO ACTION — application code nulls LinkedClaimIdFk before deleting

Implementation note

The SQL Server engine refuses to create a cascading FK on a table that already has cascading children (potential cycle). The 837↔835 self-link is therefore declared NoAction; the bulk-delete code in `ClaimsController.DeleteSelected` explicitly nulls `LinkedClaimIdFk` on every affected row in a separate `SaveChangesAsync()` round-trip before deleting.

5. Build Phases

The application was built in distinct phases, each producing a working slice. The phasing made it possible to validate each layer (parser, engine, UI) independently and avoid the big-bang integration risk that often sinks rewrite projects.

5.1 Phase 1 — Skeleton

****Output****: a runnable empty MVC application with Identity, EF Core, Bootstrap 5, and a homepage that confirms the database is reachable.

- Bootstrapped with `dotnet new mvc --auth Individual`
- Switched UI from default jQuery + bootstrap.bundle to PMTAC theme (purple navbar, icon-over-label nav)
- Connected to LocalDB; first migration creates the ASP.NET Identity tables
- Verified login/register works
- Output: a deployable shell ready for domain code

5.2 Phase 2 — Reference data loaders

****Output****: an admin-only Settings page that ingests the four reference workbooks into the database.

- **ICrosswalkLoader** — reads eMedNY APG Crosswalk .xlsx, replaces hcpcs_to_eapg + icd10_to_eapg
- **IWeightsHistoryLoader** — reads NYS DOH weights .xls (BIFF format via NPOI), populates apg_weights, px_based_weights, fee_schedule
- **IPmtacFeeCalculatorLoader** — reads PMTAC Fee Calculator .xlsm, populates apg_base_rates + provider_county
- **IDtcBaseRatesLoader** — alternative source for base rates only
- **IMasterResetService** — destructive, wipes every reference table (preserves users, audit log, claims, providers)
- Each loader is idempotent: running twice with the same file produces the same final state

5.3 Phase 3 — APG calculation engine

****Output****: the IApEngine service that converts a parsed claim DTO + provider config into an APGResult.

- Direct port of backend/engines/apg_engine.py
- Implements the priority ladder: fee schedule > Px weight > APG weight
- Handles modifiers: U6 (×1.5), multi-procedure discounting (50% off secondaries), capital add-on
- Visit-purpose ICD override for incidental E/M codes
- Returns line-by-line APGLineResult plus claim-level totals and notes
- Verified against the reference value: 99213 + E11.9 in Manhattan = \$132.09 (matches Python)
- Provided through DI as a scoped service

5.4 Phase 4 — EDI parsers and upload pipeline

****Output****: an Upload page that accepts X12 EDI 835/837 files, parses them, stores claims and lines, and prices them via the APG engine.

Parsers

- **Edi835IParser / Edi835PParser** — institutional and professional remittance parsers; both produce a `Parsed835IResult` with claim list
- **Edi837Parser** — handles both institutional (837I) and professional (837P) submissions; auto-detects from GS08 implementation guide identifier
- **EdiCommon** — segment splitter, element splitter, qualifier helpers shared across parsers

Upload simplification (later session)

- UI dropped from 4-option (835I/835P/837I/837P) to 2-option (835 / 837); the specific subtype is auto-detected
- Multi-file upload — the user picks any number of EDI files at once
- **EdiFileTypeDetector** — inspects bytes:
 - 837: GS08 implementation guide (X222 = P, X223 = I); fallback to SV1 vs SV2 segment presence
 - 835: SVC qualifier scan (NU = institutional revenue codes, HC only = likely professional); defaults to 835I in ambiguous cases

Upload service

- **ClaimUploadService** — orchestrates: parse → save → price → link
- Each parsed claim goes through the engine immediately if a provider config exists
- After a batch, **ClaimLinkerService** auto-links any 837s/835s that share the same CLP01 ID, copies dx codes from 837 → 835, and re-prices the 835

5.5 Phase 5 — Analytics and exports

****Output****: the Analytics dashboard and Excel/PDF export plumbing.

Analytics service

- **IAalyticsService.ComputeAsync(filters, ct)** — single call returns a fully-populated `AnalyticsViewModel`
- Per-user isolated via `OwnedBy(currentUser)` at the query root
- Six panels: Summary KPIs, Trends (monthly/quarterly), Denials by CARC, Top underpaid procedures, Compression breakdown (group-by EAPG/CPT/Peer/Region/Year), Payer scorecard
- All aggregations are server-side LINQ → translated to SQL by EF Core
- Time-series uses `GroupBy` on Year+Month then optionally rolls up to Quarter in memory

Exports

- **ExportService.BuildClaimsListXlsx** — list export

- **ExportService.BuildClaimDetailXlsx** — single-claim every-field export
- **ExportService.BuildFullDataXlsx** — multi-sheet workbook joinable on ClaimId/ClaimIdFk
- **ExportService.BuildAnalyticsXlsx** — 9-sheet analytics dashboard export
- **ExportService.BuildClaimDetailPdf** — narrative PDF (QuestPDF)
- **ExportService.BuildCms1500Pdf / BuildUb04Pdf** — form-shaped PDFs

5.6 Phase 6 — Polish, theming, deployment readiness

- Custom **PMTAC theme** in `wwwroot/css/pmtac-theme.css`: purple #6E50CD primary, navbar, breadcrumbs, cards, badges
- Layout rebuild — icon-over-label nav, brand block on left, bell + avatar on right
- Bulk delete on Claims list (admin/analyst only) with checkboxes, master select-all, and per-row delete
- Status messages via TempData on delete
- Per-row CMS-1500 / UB-04 / Excel / Full Data export buttons on each claim
- Smoke-test fixtures: `samples/paired_835i.edi` and `samples/paired_837i.edi` demonstrating the linker + visit-purpose override pathway

5.7 Post-go-live extensions

After the initial six phases shipped, several feature extensions were added based on real use feedback. These are the same code in the same project, just delivered later.

5.7.1 User management + role-based authorization

- Three roles: **admin**, **analyst**, **viewer**
- Admin-only Users page (`/Users`) — list, create, edit role, reset password, delete
- Self-service registration blocked via Razor Pages convention; bootstrap-aware policy lets the very first registration succeed when no users exist
- Safety guards: cannot demote yourself if sole admin, cannot lock yourself out, cannot delete the last admin

5.7.2 Per-user data isolation

- Added OwnerUserId (string FK to `AspNetUsers.Id`) to `ParsedClaim` and `ProviderConfig`
- **ICurrentUserContext** + **OwnedQueryExtensions.OwnedBy(ctx)** — every read query gets a one-line filter
- Migration backfilled all existing rows to the seeded admin
- **View-as switcher** in navbar (admin/viewer only) — scope the session to one user; persistent breadcrumb makes it obvious
- Owner column on the Claims list, visible only when admin/viewer is in unscoped (all-users) mode

5.7.3 CMS Medicare PFS integration

- **CmsRateService** — port of Python `cms_engine.py`

- Self-healing UUID resolution from pfs.data.cms.gov/data.json
- Two-dataset model: "Indicators for YYYY" (RVUs + CF) + "Localities for YYYY" (GPCIs)
- Formula: $\text{payment} = ((\text{rvu_work} \times \text{gpci_w}) + (\text{pe_rvu} \times \text{gpci_pe}) + (\text{rvu_mp} \times \text{gpci_mp})) \times \text{CF}$
- Optional PC/TC split (-26 professional, -TC technical)
- 24-hour DB cache + graceful stale-cache fallback when CMS unreachable
- Wired into the Rate Calculator (rate-source dropdown) and the 837P/835P claim Detail page (auto comparison panel)
- Admin-only Settings card to invalidate the cache after CMS quarterly publications

5.7.4 Multi-line Rate Calculator

- JS-driven add-row / remove-row UI — no fixed maximum
- Rate-source dropdown: APG (NYS Institutional) / CMS Medicare (Professional) / Both side-by-side
- Searchable CMS locality dropdown grouped by MAC region
- Facility vs non-facility toggle, PC/TC split toggle

5.7.5 Analytics dashboard

- Filter bar: date range (default last 12 months), payer, file type, NPI, group-by, trend period
- Six panels: KPI tiles, Trends line chart, Denials by CARC bar chart, Top underpaid procedures, Compression breakdown, Payer scorecard
- Drill-through links from procedure / payer / claim ID rows back to filtered Claims list
- Excel export — 9-sheet workbook honoring all current filters

6. Security and Access Control

6.1 Authentication

Authentication is handled by ASP.NET Core Identity with default cookie-based sessions. Passwords are hashed with PBKDF2 + per-user salt (Identity defaults). The cookie is HTTP-only, secure, and SameSite=Lax. There is no refresh-token flow because there is no separate API.

6.2 Authorization — role matrix

Capability	Admin	Analyst	Viewer
View Dashboard, Claims, Analytics, Calculator	✓	✓	✓
Upload claim files	✓	✓	—
Delete claims (single + bulk)	✓	✓	—
Export to Excel / PDF / Full Data	✓	✓	✓
Edit Provider Configuration	✓	✓	—
Reference-data Settings (upload workbooks, master reset, refresh CMS)	✓	—	—
User management (create, edit role, reset password, delete)	✓	—	—
View-as switcher (scope session to one user's data)	✓	—	✓

6.3 Role enforcement

Roles are enforced at controller-action granularity using `[Authorize(Roles = "...")]`:

C#

// Admin only
[Authorize(Roles = "admin")]
public class UsersController : Controller { ... }
// Editor roles (admin OR analyst)
[Authorize(Roles = RoleSeeder.EditorRoles)]
public class UploadController : Controller { ... }
// Any authenticated user
[Authorize]
public class ClaimsController : Controller { ... }

6.4 Per-user data isolation

Even though analysts and viewers share a database, their **visible** data is scoped to themselves unless they have an admin/viewer role and explicitly switch context. The mechanism:

6. **Stamping on writes** — `ClaimUploadService` and `ProviderConfigController`.`Save` set `OwnerUserId = currentUser.SignedInUserId` on every newly-created row. The signed-in user is used here, **not** the view-as target — admins can't accidentally upload into another user's bucket while scoped.
7. **Filtering on reads** — every controller that reads `ParsedClaim` or `ProviderConfig` pipes through `db.ParsedClaims.OwnedBy(currentUser)`. The extension method applies `WHERE OwnerUserId = ...` based on the effective owner filter.
8. **Effective owner** — `ICurrentUserContext.EffectiveOwnerFilter` returns `NULL` (no filter, see everything) for admins/viewers in unscoped mode, otherwise the signed-in or view-as target user ID.
9. **Sibling/linker scoping** — `ClaimLinkerService.LinkAndEnrichAsync` takes an `ownerUserId` parameter, so two analysts who both upload claims with the same CLM01 won't cross-link.

C# — `OwnedQueryExtensions.cs`

<code>// Marker interface; implemented by ParsedClaim and ProviderConfig</code>
<code>public interface IOwnedByUser</code>
<code>{</code>
<code> string? OwnerUserId { get; set; }</code>
<code>}</code>
<code>// Extension applied at every read site</code>
<code>public static IQueryable<T> OwnedBy<T>(this IQueryable<T> query,</code>
<code> ICurrentUserContext ctx)</code>
<code> where T : class, IOwnedByUser</code>
<code>{</code>
<code> var owner = ctx.EffectiveOwnerFilter;</code>
<code> if (owner is null) return query; // unscoped admin/viewer – see everything</code>
<code> return query.Where(e => e.OwnerUserId == owner);</code>
<code>}</code>

6.5 The bootstrap-admin problem

A fresh deployment has zero users and zero admins. With self-service registration blocked, no one can create the first admin — chicken-and-egg. The application solves this with an

****AdminOrBootstrapRequirement**** authorization handler:

- If the current user is in the admin role → succeed (the normal path post-bootstrap)
- Else if there are zero users in the entire database → succeed (bootstrap mode for the first deploy)

- Otherwise → fail (the default for any non-admin trying to register)

After the first user registers, the app must be restarted (Azure Portal → App Service → Restart) so that `RoleSeeder.SeedAsync`` runs again and promotes the first user to admin. From that point forward the bootstrap branch is unreachable and registration is admin-only.

7. CMS Medicare PFS Integration

The CMS Medicare integration computes the equivalent professional-services payment under the CMS Physician Fee Schedule (PFS). It is used in three places: the Rate Calculator (manual CPT entry), the 837P/835P claim Detail page (automatic comparison panel), and the planned analytics module that benchmarks Medicaid against Medicare.

7.1 Data source

- **Endpoint:** `https://pfs.data.cms.gov/data.json` (catalog) and `/api/1/datastore/query/{uuid}/0` (DKAN POST queries)
- **No authentication** — public, anonymous CMS data
- **Two datasets per year:** "Indicators for YYYY" (RVUs + conversion factor) and "Localities for YYYY" (GPCIs)
- **Self-healing UUID resolution** — UUIDs are *never* hardcoded; the engine fetches `data.json`, regex-matches "Indicators for {year}{suffix}" and "Localities for {year}{suffix}", and picks the newest suffix (preference: no suffix > B > A)

7.2 Formula

CMS PFS formula

non_facility_rate = ((rvu_work × gpci_work)	
+ (pe_rvu_nonfac × gpci_pe)	
+ (rvu_mp × gpci_mp)) × conversion_factor	
facility_rate = ((rvu_work × gpci_work)	
+ (pe_rvu_facility × gpci_pe)	
+ (rvu_mp × gpci_mp)) × conversion_factor	

- **Non-facility rate** — applies in independent clinics / offices
- **Facility rate** — lower; applies in hospital outpatient settings (CMS pays the facility separately)
- **PC/TC split** (optional) — fetches the -26 (professional component) and -TC (technical component) modifier rows in parallel; not every HCPCS has these

7.3 Caching strategy

Three layers, in order of speed:

10. **In-process catalog cache** (24-hour TTL, per process) — the fetched `data.json` plus computed UUIDs for each requested year. First request hits CMS; subsequent requests are O(1).
11. **In-process locality-list cache** (24-hour TTL) — populated when the locality dropdown is rendered. ~110 rows per year.
12. **Persistent DB cache** — `cms_rate_cache` table, keyed by (HCPCS, modifier, locality, year). 24-hour `CachedUntil` timestamp. Stale rows are still returned if the live API is unreachable (graceful degradation).

7.4 Cache refresh

Admins can invalidate the cache via Settings → "Refresh CMS fee schedule cache". The button:

- Runs a single SQL UPDATE `cms_rate_cache SET CachedUntil = utcnow` (instant, no API calls)
- Clears the in-process catalog and locality caches
- Subsequent rate lookups will re-fetch fresh data from CMS
- Designed for the quarterly CMS PFS publication cadence (Jan/Apr/Jul/Oct, plus occasional mid-year corrections)

7.5 Outbound network requirement

IT note

Production deployment requires **outbound HTTPS to pfs.data.cms.gov** through any corporate firewall or NSG. The beta deployment confirmed Azure App Service's default outbound ruleset allows this. If your enterprise environment restricts outbound traffic, add this domain to the allow-list before deployment, or expect "CMS catalog unreachable" banners in the UI.

7.6 Threading caveat

EF Core's `DbContext` is **not thread-safe** — only one query may be in flight per instance at a time. Early in development, the calculator code launched the base, -26, and -TC cache lookups via `Task.WhenAll` to parallelize. On Azure SQL Free Tier (which has higher latency than LocalDB), the second concurrent query would start before the first completed, triggering:

EF Core

"A second operation was started on this context instance before a previous
operation completed. This is usually caused by different threads
concurrently using the same instance of DbContext."

The fix was to make the cache reads sequential. The CMS HTTP fetches inside `CmsRateService` still parallelize (using `Task.WhenAll` over independent `HttpClient` calls), so the user-facing latency is unchanged. The serialized work is just the per-row DB cache lookup, which is a B-tree seek on a unique index — sub-millisecond.

8. Azure Trial Deployment

This section is the playbook used to stand up the beta environment on Azure free-tier resources. Total monthly cost: \$0 (with caveats — see Section 8.10). Total deployment time: ~60 minutes including Azure account creation, with another ~30 minutes for first-time troubleshooting of common cold-start issues.

8.1 Prerequisites

- A working .NET 10 build of the application (verified by `dotnet build` exiting clean)
- Visual Studio 2022 17.x or later with the "Azure development" workload installed
- A Microsoft account (work or personal) — used for Azure sign-in
- A credit card (used for Microsoft's identity verification only; the free SKUs we use never bill it)
- Outbound HTTPS allowed from your laptop to `portal.azure.com` and `pfs.data.cms.gov`
- LocalDB running and the app verified working in dev mode (so we know the build is good before we deploy it)

8.2 Code-level preparation

Before publishing, three small code-level changes ensure the deployed app behaves correctly:

13. **Move the dev connection string out of `appsettings.json`** into `appsettings.Development.json`. The production-loaded `appsettings.json` should have NO connection string — it will be supplied by Azure App Service Configuration.
14. **Add `appsettings.Production.json`** with sensible production logging defaults (Warning level for noisy namespaces, Information for APGAnalyzer).
15. **Confirm `Program.cs` reads the connection string via `builder.Configuration.GetConnectionString("DefaultConnection")`** — this naturally picks up Azure's injected value.

JSON

<code>// appsettings.json (production-loaded)</code>
<code>{</code>
<code> // ConnectionStrings:DefaultConnection is intentionally NOT set here.</code>
<code> // Dev: loaded from appsettings.Development.json (LocalDB).</code>
<code> // Prod: loaded from Azure App Service > Configuration</code>
<code> // > Connection strings (the Publish wizard provisions this).</code>
<code> "Logging": { ... },</code>
<code> "AllowedHosts": ""</code>
<code>}</code>

8.3 Step C1 — Create the free Azure SQL Database

The free Azure SQL Database tier (announced 2024) provides 32 GB storage, 100,000 vCore-seconds/month, and a 1-hour auto-pause delay. It is intentionally provisioned **before** the App Service so we can wire up the connection string in one step during the App Service creation.

16. Sign into the Azure Portal at portal.azure.com.
17. Search for "Azure SQL Database" in the top search bar; click the result with type "SQL server" (the Microsoft.Sql/servers/databases resource).
18. On the SQL databases listing page, click + **Create** → **SQL database (Free offer)**. The free-offer flow pre-configures everything for you.
19. On the **Basics** tab, fill in:
 - · Subscription: Azure subscription 1
 - · Resource group: create new, name it apg-analyzer-beta
 - · Database name: APGAnalyzer
 - · Server: create new (see next step)
 - · Workload environment: Development
20. In the "Create new server" panel:
 - · Server name: e.g., apg-analyzer-sql-pmtac (must be globally unique; add suffix if taken)
 - · Location: try Central US first; if free-tier capacity is exhausted, try Canada Central, then East US 2 / West US 2 / South Central US in turn
 - · Authentication method: SQL authentication
 - · Server admin login: sqladmin
 - · Password: generate a strong password and **save it in a password manager immediately** — Microsoft does not show it again
21. Confirm "Free offer applied" banner shows on the form (100k vCore-sec, 32 GB data, 32 GB backup) and Estimated total = Free.
22. Click **Review + create** → **Create**. Provisioning takes 2-5 minutes.
23. On the deployment-complete page, click **Go to resource**. Confirm Status = Online.

Free-tier capacity quirk

Microsoft caps how many free databases each region hosts. New subscriptions sometimes cannot create a free database in popular regions (East US, West US). Cycle through Central US → Canada Central → East US 2 → West US 2 → South Central US until one accepts. Canada Central is fully HIPAA-compatible under Microsoft's Canada BAA if US regions are unavailable.

8.4 Step C2 — Create the App Service

The Visual Studio Publish wizard does most of this in one flow. Two gotchas: VS sometimes hides the F1 free tier from the Hosting Plan size dropdown; if that happens, create the App Service manually in the Portal first (Portal always shows F1) and pick "existing" in the wizard.

24. In Visual Studio Solution Explorer, right-click **APGAnalyzer** → **Publish**.
25. On the first wizard screen pick **Azure** → **Next**.
26. On the second screen pick **Azure App Service (Windows)** → **Next**.
27. Sign into Azure with the same Microsoft account.

28. Click **+ Create new** to create a new App Service. Fill the form:
 - · Name: `apg-analyzer-beta-pmtac` (becomes part of the URL)
 - · Resource group: pick `apg-analyzer-beta` (the one created with the SQL DB)
 - · Hosting Plan: New → name `apg-analyzer-plan`, location **Central US** (must match SQL DB), Size **F1 Free**
29. If F1 doesn't appear in the Size dropdown: cancel out, go to Azure Portal → App Services → +Create → Web App, pick F1 there directly. The Portal's tier picker is more complete.
30. Click **Create** in the wizard, wait ~60-90 seconds for provisioning.
31. Back on the Publish target list, select the new app, leave "Deploy as ZIP package" checked, leave "Turn on Basic Authentication" unchecked. Click **Finish**.
32. On the resulting Publish summary page, verify Configuration = Release, Target Framework = `net10.0`, App Service = `apg-analyzer-beta-pmtac`.
33. Click **Publish** at the top right. Build + ZIP + upload + restart takes 3-5 minutes. Browser opens automatically when done.

8.5 Step C2b — Wire up the connection string

The deployed app needs to know how to reach the SQL database. We add the connection string to App Service Configuration as an environment variable.

34. In Azure Portal, navigate to your **APGAnalyzer** SQL database → **Connection strings** in the left sidebar.
35. Copy the **ADO.NET (SQL authentication)** string (the bottom box). It looks like: `Server=tcp:apg-analyzer-sql-pmtac.database.windows.net,1433;Initial Catalog=APGAnalyzer;User ID=sqladmin;Password={your_password};...`
36. Open Notepad, paste the string, replace `{your_password}` with the real password (no curly braces).
37. Navigate to your **apg-analyzer-beta-pmtac** App Service → **Settings** → **Environment variables** → **Connection strings** tab.
38. Click **+ Add**. Fill in:
 - · Name: `DefaultConnection` (case-sensitive; must match exactly what Program.cs reads)
 - · Value: paste the full string from Notepad
 - · Type: **SQLAzure**
39. Click **Apply** in the popup, then **Apply** at the top of the page (Azure has a two-step save). Confirm the app restart.

8.6 Step C2c — SQL firewall rules

The SQL Server has a firewall that defaults to deny-all. Two rules need to exist:

- **Allow Azure services and resources** — let the App Service connect (this is usually enabled automatically by the Free offer creation flow)
- **Your laptop's public IP** — required to apply migrations from your local PowerShell using `dotnet ef database update`

40. Navigate to your SQL **server** (not the database) → **Security** → **Networking**.
41. Confirm "Allow Azure services and resources to access this server" = **Yes**.
42. Click + **Add your client IPv4 address (xxx.xxx.xxx.xxx)** — Azure auto-detects your IP.
43. Click **Save** at the top of the page.

8.7 Step C3 — Apply database migrations

EF Core migrations create all the schema (Identity tables + reference tables + operational tables + cms_rate_cache). They run from your laptop against the Azure SQL database, not from the App Service.

44. Open a non-admin Windows PowerShell window (LocalDB instances are scoped per Windows user, so the admin shell sees a different LocalDB universe than your normal session — wrong shell will produce confusing errors).
45. Navigate to the project: `cd "H:\Working\Projects\APG Calculator DotNet\APGAnalyzer"`
46. Copy the connection string (with the real password) from your Notepad to clipboard.
47. Capture the clipboard into a variable: `$conn = Get-Clipboard`
48. Sanity check: `$conn.Length` (should be ~250-280 characters)
49. Apply migrations: `dotnet ef database update --connection $conn`
50. Wait 1-3 minutes. EF Core builds the project, connects, runs all migrations in order. Final line: Done.

Azure SQL Free Tier auto-pauses after 1 hour idle. The first connection of the day fails because the DB is still waking. Fix: open the SQL database overview in the Portal (which triggers a wake-up), wait until Status = Online, then immediately retry the migration. The DB takes ~30-60 seconds to come back. This same error pattern reappears throughout the trial deployment whenever the DB has been idle for 1+ hour — see Section 9.2 for ongoing operational handling.

8.8 Step D — Bootstrap the first admin

The very first user who registers on a fresh deployment gets auto-promoted to admin via the ``AdminOrBootstrapRequirement`` policy + ``RoleSeeder.SeedAsync`` startup code.

51. Navigate to your live URL: `https://apg-analyzer-beta-pmtac-{random}.centralus-01.azurewebsites.net`
52. Wait 30-90 seconds for first-request cold start.
53. Click **Login** → "Register as a new user" link
54. Register with your real email and a strong password. After successful registration you'll be auto-signed-in but **without** any role yet.
55. Restart the App Service from the Portal: navigate to the App Service → **Restart** at the top toolbar → confirm. This forces `RoleSeeder.SeedAsync` to run again. It detects "1 user, 0 admins" and promotes you.

56. Wait ~60 seconds for the App Service to come back up.
57. Sign back in. Your navbar should now show admin-only items: Provider, Users, Settings, plus the "Viewing: All users" view-as dropdown in the top-right.

8.9 Step D2 — Load reference data and create beta accounts

From the admin dashboard:

58. Click **Settings** in the navbar (admin-only).
59. Upload the four reference workbooks **in order**:
 - Card 1 — eMedNY APG Crosswalk (.xlsx) → ~96k rows, ~30 sec
 - Card 2 — NYS DOH Weights + Fees (.xls) → ~29k rows, ~3-5 sec
 - Card 3 — PMTAC Fee Calculator (.xslm) → ~250 rows, ~5 sec
 - Card 4 — DTC Base Rates (.xls, optional) → 182 rows
60. Verify on the Dashboard: Total reference rows ≈ 125,249.
61. Set up your own Provider Config (Provider in navbar) with peer group, county, region, CMS locality (e.g., 1320201 for Manhattan).
62. Create beta tester accounts via Users → + Add user. For each tester pick the appropriate role (analyst for daily users, viewer for oversight roles).
63. Send each tester their URL, login email, and temp password through your normal corporate channel. Recommend they change their password on first login.

8.10 Step D3 — Smoke tests

Three quick verifications to run before letting beta testers loose:

64. **APG path** — Rate Calculator with source=APG, principal ICD = E11.9, CPT = 99213, locality = whatever matches your peer group + county. Expected result: ~\$132.09 (matches the canonical reference).
65. **CMS path** — Rate Calculator with source=CMS Medicare, locality = 1320201, CPT = 99213. Expected: ~\$107.63 (Manhattan 2026, varies by year as CMS publishes updates). This verifies outbound HTTPS to pfs.data.cms.gov from Azure App Service.
66. **Upload pipeline** — upload one of the synthetic samples (samples/paired_835i.edi or paired_837i.edi). Verify the claim appears on the Claims list, the calculation completes, and the Detail page renders.

9. Operational Notes

9.1 Free-tier characteristics and trade-offs

Tier	Cost / month	Auto-sleep	CPU cap	RAM	Best for
F1 (App Service Free)	\$0	20 min idle	60 min/day	1 GB shared	Demo, light testing, single-user
B1 (App Service Basic)	~\$13	never	no cap	1.75 GB dedicated	Active beta with real upload workloads
SQL Free Tier	\$0	1 hour idle	100k vCore-sec/month	32 GB storage	Beta with infrequent use
SQL Basic (\$5/mo)	~\$5	never	~5 DTU	2 GB storage	Always-warm beta or low-traffic prod
SQL S0 / S1	\$15-30	never	higher DTU	250 GB storage	Light production

9.2 Cold-start playbook

On the all-free configuration (F1 + free SQL), both layers can be asleep at once. The first request after >1 hour idle hits this sequence:

67. Browser → App Service → cold-start the worker process (~30-60 sec)
68. App Service → SQL DB → DB resume from auto-pause (~30-60 sec)
69. App startup → RoleSeeder.SeedAsync → first DB query (~5 sec)

Combined first-request time: ****60-120 seconds****. Subsequent requests (and any request within 20 min of the previous one) are fast.

****If a request times out (HTTP 500.30) after a long idle period**:**

70. Open the SQL database in Azure Portal — viewing the Overview page triggers a wake-up.
71. Wait until Status = Online (refresh the page every 15 sec).
72. Open the App Service in Azure Portal → click **Restart** at the top.
73. Wait ~90 seconds.
74. Refresh your browser tab (Ctrl+F5).

****To eliminate cold starts permanently**:** scale up to App Service B1 (~\$13/mo) and SQL Basic (~\$5/mo). Both stay warm 24/7. Beta testers will see snappy first-load, the experience matches production. Total ~\$18/mo for the entire beta period.

9.3 Quarterly CMS refresh

CMS publishes new PFS data ~quarterly (Jan, Apr, Jul, Oct, plus mid-year corrections). When the new dataset is up:

75. Sign in as admin.
76. Settings → Card 5: **Refresh CMS fee schedule cache**.
77. Confirm the dialog.

The button: clears in-process caches (catalog, locality lists), runs ``UPDATE cms_rate_cache SET CachedUntil = utcnow`` (instant). Subsequent rate lookups will pull fresh data from CMS. The next user to use the calculator pays the cold-fetch cost (~5 sec); everyone after that hits the new warm cache.

9.4 Reference data refresh

When NYS DOH publishes new APG reference data (typically annually or after methodology updates):

78. Download the four PMTAC / NYS DOH reference workbooks for the new period.
79. Sign in as admin.
80. Settings → upload the four files via Cards 1-4. Each upload **replaces** the previous data (not append).
81. Verify Dashboard totals match expected counts.
82. No claims data is touched; the engine's next calculation will use the new reference data.

9.5 Backups

- Azure SQL Database has **automatic point-in-time-restore** included on every tier (7 days for Free/Basic, 35 days for S0+).
- Long-term backups can be configured at SQL Server → Backups → Retention policies.
- App Service code is recoverable from the local Visual Studio publish profile and git — no separate App Service backup is required.
- Reference data workbooks should be archived in PMTAC's document store, not relied on as "live" inside the application.

10. Roadmap and Deferred Items

10.1 Cosmetic / quick wins

- **Footer text** says "SQL Server (LocalDB)" and "Phase 1 — Skeleton" — both are stale strings from early dev. ~10 min cleanup.
- **Application Insights** — free tier is sufficient for beta-scale usage. Adds error tracking, slow-request analysis, and dependency telemetry. ~5 min wire-up via Azure Portal → App Service → Application Insights.
- **Custom domain** — instead of `apg-analyzer-beta-pmtac-xxx.azurewebsites.net`, point a friendly subdomain like `apg-beta.pmtac.com` at the App Service. Requires DNS access plus an SSL certificate (free via App Service Managed Cert).

10.2 Feature work deferred from V1

- **ZIP → CMS locality auto-resolution** — the Python build had a `zip_locality` table with the CMS ZIP5 file imported. The C# port relies on the locality dropdown instead. Quick to add when needed; the file is ~50k rows.
- **Saved analytics filter presets per user** — would need a small new table (`analytics_preset` with `OwnerId` + `name` + `filter JSON`) plus CRUD UI. Defer until usage tells us which filter combos people save vs. retype.
- **Bulk re-pricing of historical claims against CMS** — currently claims are priced APG-only at upload time. A "compute CMS variance for all claims" job would let the analytics dashboard show APG-vs-CMS comparisons retroactively.

10.3 V3 — separate analytics application

The current analytics page (V1 Tier 1+2+3) is intentionally simple: server-rendered Razor + Chart.js, no SPA framework. For deeper analytics (Days in A/R, Collection Rates, E&M Bell Curve, drill-through, saved dashboards), the proposal is a ****separate ASP.NET Core MVC project**** (working name ``APGAnalyzer.V3``) in the same Git repo and same Azure SQL database, but with its own deployment and Chart.js + DataTables stack. This separation:

- Keeps the V1 surface stable while V3 evolves
- Lets V3 use a richer JS framework (React or Vue) without rewriting V1
- Enables deploying V3 to a different App Service tier (e.g., heavier compute) than V1
- Reuses the same Identity / per-user isolation / claims data

10.4 Production deployment (handed to IT)

When IT does the production Azure deployment, they should:

- Use the same migration set (no schema differences between beta and prod)
- Pick **B1 or higher** App Service tier and **S0 or higher** SQL DB tier — production-like behavior, no cold-start issues, larger backup retention

- Confirm outbound HTTPS to **pfs.data.cms.gov** is allowed by NSG / firewall rules (verified working in beta on standard Azure outbound)
- Configure the connection string via App Service Configuration (NEVER in `appsettings.json`)
- Wire up Application Insights for production monitoring
- Consider Azure Front Door / Web Application Firewall in front of the App Service if exposing the application beyond the corporate network
- Coordinate the bootstrap-admin step (first registered user becomes admin) with the IT-provisioned account
- Decide on a reference-data refresh policy and document it for ongoing operations

11. Appendices

11.1 Glossary

Term	Definition
APG / EAPG	Ambulatory Patient Group / Enhanced APG — a payment classification system for outpatient services
Article 28	New York State Public Health Law section that licenses outpatient clinics, including diagnostic & treatment centers (DTCs)
CARC	Claim Adjustment Reason Code — explains why a payer reduced or denied a payment (e.g., 96 = "Non-covered charge")
CLP / CLM	X12 EDI segment — claim payment information (835) / claim information (837)
CPT / HCPCS	Current Procedural Terminology / Healthcare Common Procedure Coding System — the standard procedure-code vocabulary
CMS	Centers for Medicare & Medicaid Services — federal agency that publishes Medicare reimbursement rates
DKAN	Open-source datastore platform CMS uses to publish public data; underpins pfs.data.cms.gov
DTC	Diagnostic & Treatment Center — Article 28-licensed outpatient clinic in New York
EDI / X12	Electronic Data Interchange — the file format for medical claim transmission, governed by ASC X12
GPCI	Geographic Practice Cost Index — multipliers in the CMS PFS formula reflecting regional cost differences
ICD-10	International Classification of Diseases v10 — diagnosis code vocabulary
MAC	Medicare Administrative Contractor — regional CMS contractor processing Medicare claims
MPFS / PFS	Medicare Physician Fee Schedule — CMS's rate table for professional services
NPI	National Provider Identifier — 10-digit ID for healthcare providers
NYS DOH	New York State Department of Health — publishes the APG methodology and rates
PCT / TC	Professional Component (-26 modifier) and Technical Component (-TC modifier) of a procedure (e.g., radiology)

Term	Definition
RVU	Relative Value Unit — components in the CMS PFS formula: work, practice expense (PE), malpractice (MP)
U6 modifier	NYS-specific modifier indicating an after-hours service; multiplies the APG payment by 1.5×
UB-04 / CMS-1500	Standard claim form layouts: UB-04 for institutional, CMS-1500 for professional
VS Publish wizard	Visual Studio's built-in deploy-to-Azure tool — generates a publish profile (.pubxml) and pushes a ZIP package

11.2 Key file locations

Path	What's there
APGAnalyzer/Program.cs	Composition root; service registrations; HTTP pipeline; RoleSeeder boot
APGAnalyzer/Controllers/	MVC controllers — one per feature area
APGAnalyzer/Views/	Razor views — folder per controller, plus Shared/
APGAnalyzer/Models/	View models, filter models, domain DTOs
APGAnalyzer/Models/Domain/	EF Core entities — one class per table
APGAnalyzer/Models/Engine/	Engine input/output DTOs (ParsedClaimDto, APGResult, APGLineResult)
APGAnalyzer/Services/	Business services — engine, parsers, exporters, loaders
APGAnalyzer/Services/Edi/	EDI parsers + EdiFileTypeDetector
APGAnalyzer/Services/Cms/	CmsRateService and supporting types
APGAnalyzer/Data/Migrations/	EF Core migrations (chronological)
APGAnalyzer/wwwroot/css/pmtac-theme.css	PMTAC custom theme overrides
APGAnalyzer/wwwroot/images/pmtac-logo.png	Brand logo (38px tall, transparent PNG)
APGAnalyzer/Properties/PublishProfiles/	Saved Publish wizard profiles (per environment)
samples/paired_835i.edi, paired_837i.edi	Synthetic test fixtures for the linker pathway
docs/AZURE_SQL_SCHEMA.sql	Standalone T-SQL DDL of the entire production schema (for IT review)

Path	What's there
docs/APGAnalyzer_Technical_Documentation.docx	This document

11.3 Standard error references

Symptom	Most likely cause	Fix
HTTP 500.30 ASP.NET Core app failed to start	DB connection failing during RoleSeeder	Wake DB in Portal, then restart App Service
Migration error 40613 "database not currently available"	SQL Free tier auto-paused	Open DB overview to wake; retry migration
Migration error 40615 "client IP not allowed"	Your laptop IP not in SQL firewall	Add IP via SQL Server → Networking → Firewall rules
EF "second operation on this context" exception	Parallel queries on shared DbContext	Make queries sequential or inject DbContextFactory
HTTP 500.30 immediately after upload	F1 OOM during parse + re-pricing	Restart App Service; consider scaling to B1
Calculator "CMS catalog unreachable" banner	Outbound HTTPS to pfs.data.cms.gov blocked	Verify NSG / firewall rules; cached values still serve
F1 quota exceeded after heavy use	60 min/day CPU cap hit	Wait until midnight UTC, or scale to B1
App responsive but stuck on old data	In-process cache (CMS, locality)	Settings → Refresh CMS fee schedule cache; or restart App Service

11.4 Useful Azure CLI commands

PowerShell / Azure CLI

Wake the SQL database manually (forces resume from auto-pause)
az sql db show \
--resource-group apg-analyzer-beta \
--server apg-analyzer-sql-pmtac \
--name APGAnalyzer
Restart the App Service
az webapp restart \
--resource-group apg-analyzer-beta \
--name apg-analyzer-beta-pmtac

Tail live application logs
az webapp log tail \
--resource-group apg-analyzer-beta \
--name apg-analyzer-beta-pmtac
Scale up to B1 (production-grade)
az appservice plan update \
--resource-group apg-analyzer-beta \
--name apg-analyzer-plan \
--sku B1
Apply EF migrations from local machine to Azure SQL
\$conn = Get-Clipboard
dotnet ef database update --connection \$conn

11.5 Document revision history

Version	Date	Author	Notes
1.0	May 2026	Project documentation	Initial release covering V1 + Azure beta deployment